

CCS 3105: Systems Programming

Week 4 Lab Report

Linux File Systems, Directories and File Management

Name: David Mujabi

Reg. No.: C026-01-0802/2024

Date: 06/08/2026

0.1. 1. Objectives

The aim of this session was to become comfortable working with the Linux file system from the terminal rather than a GUI. By the end of the practical I was expected to navigate the directory tree using absolute and relative paths, create and reorganise directories and files, and interpret and change file permissions and ownership. I also needed to understand what an inode is and how it explains the behaviour of hard and symbolic links, and to use the standard search and inspection tools such as find, grep, stat, df and du.

0.2. 2. Procedure

I started by checking where I was with `pwd` and listing the contents of `/` with `ls`, then moved to the root with `cd /`, returned home with `cd ~`, and briefly explored `/etc` to see configuration files. Next I built the required directory tree by creating `Week4` and its subdirectories `Documents`, `Programs/C`, `Programs/Backup` and `Reports` using `mkdir`, and confirmed the structure with `ls -R`. I then removed `Programs/Backup` with `rmdir` since it was empty.

For file management I used `touch` to create `report.txt`, `marks.txt` and `students.txt`, then renamed `report.txt` to `report2026.txt` with `mv`, copied it as `copy.txt` with `cp`, deleted the copy with `rm`, and moved `students.txt` into `Documents`. I checked permissions with `ls -l` before and after running `chmod 755 report2026.txt` and then `chmod 600 report2026.txt`, and inspected ownership with `whoami` and `groups`.

To study links I created `original.txt`, made a hard link with `ln original.txt hardlink.txt` and a symbolic link with `ln -s original.txt softlink.txt`, and compared the three entries with `ls -li`. After deleting the original with `rm` I tried to read both links. Finally I practised searching with `find . -name "*.txt"`, which `gcc` and `whereis gcc`, searched file contents with `grep` on `students.txt`, and inspected metadata and disk usage with `stat`, `ls -li`, `df -h` and `du -sh Week4`.

0.3. 3. Results

The table below records the outputs I observed.

Task	Command	Result
Change permissions	<code>chmod 755 report2026.txt</code>	<code>-rwxr-xr-x</code> (owner <code>rwx</code> , group/others <code>r-x</code>)
Make private	<code>chmod 600 report2026.txt</code>	<code>-rw-----</code> (only owner <code>r/w</code>)

Task	Command	Result
Inspect inodes	<code>ls -li</code>	<code>original.txt</code> and <code>hardlink.txt</code> share one inode number; <code>softlink.txt</code> has its own
Delete original	<code>rm original.txt</code>	<code>cat hardlink.txt</code> still prints content; <code>cat softlink.txt</code> fails with “No such file”
Search files	<code>find . -name "*.txt"</code>	Lists every <code>.txt</code> file under the current tree
Count matches	<code>grep -c Mary students.txt</code>	2 (Mary appears twice)
Metadata	<code>stat report2026.txt</code>	Shows size, 600 mode, owner, timestamps, inode number
Disk space	<code>df -h / du -sh Week4</code>	<code>df</code> shows free space per filesystem; <code>du</code> shows the total size of <code>Week4</code>

0.4. 4. Discussion

The most interesting result was the link experiment. Before deletion, `original.txt` and `hardlink.txt` showed the **same** inode number, while `softlink.txt` showed a different one. This is the whole point of the two link types. A hard link is not a copy and not a shortcut; it is simply a second directory entry pointing at the same inode. A file therefore does not really own its data — an inode is deleted only when its link count reaches zero. So when I ran `rm original.txt`, the kernel removed one name and decremented the link count from two to one, but the data blocks stayed intact, which is why `hardlink.txt` kept working. A symbolic link, on the other hand, is a small file of its own whose content is just the **path** to the target. Deleting the target does not touch the symlink, but the symlink now points at nothing, which is why `cat` reported “No such file”. This experiment made the difference between “names pointing to data” (hard links) and “names pointing to names” (symbolic links) concrete rather than theoretical. It also explains the restrictions: a hard link cannot span filesystems or point at a directory, because both rely on the inode table, whereas a symlink can because it only stores text.

The permission outputs also connected directly to the concept of access control. `chmod 755` produced `-rwxr-xr-x`: the owner gets read/write/execute, everyone else read/execute. Because the bits encode 4 (read), 2 (write) and 1 (execute), $7 = 4+2+1$, $5 = 4+1$, so the number and the letter form are the same thing in different notation. `chmod 600` restricted the file so that only I could read or modify it — appropriate for something like a key file. The third character in each group is the execute bit, and this matters: the kernel refuses to run a program whose execute bit is unset, so even a correct binary is useless without it. The `ls -l` output always showed ten characters: a type flag (`-` for regular files, `l` for symlinks) followed by three permission triplets for owner, group and others. Combined with `whoami` and `groups`, this is how Linux makes an access decision — the kernel first matches the process’s user ID, then its group, and falls back to “others” only if neither matches.

The searching and inspection tools each had a distinct role, and confusing them was easy to avoid only after testing. `find` walks the actual directory tree and can match on name, type, size or date, so it is the tool for locating files anywhere under a directory. `which` does something

much smaller: it looks up a command's path inside `$PATH`, so it answers “which copy would run if I typed this?”. `whereis` goes further and finds the binary together with its man page and source. `grep` is different again — it searches the **contents** of files rather than their names. Its `-i` flag made the match case-insensitive, and `-c` turned the search into a count, which is why `grep -c Mary` reported 2 even though I had typed Mary only once — it appears twice in the file, once at the start and once at the end. Finally, `stat` showed the metadata stored in the inode (size, mode, ownership, three timestamps and the inode number), and the pair `df` and `du` answer different questions: `df` reports space across whole mounted filesystems, while `du` recursively sums the size of one directory. Together these tools are the everyday vocabulary of systems programming on Unix.

0.5. 5. Challenges Encountered

My first attempt at removing `Programs/Backup` with `rmdir` failed with “directory not empty” because I had placed a file in it while testing. I solved this by deleting the file first with `rm` and then removing the now-empty directory, which also taught me that `rmdir` refuses to delete non-empty directories as a safety measure. I also briefly got a “No such file or directory” when reading the soft link after deleting the original and had to stop and think through the inode explanation to confirm this was expected behaviour rather than a mistake.

0.6. 6. Lessons Learned

The main thing I will remember is that a hard link and its original are not two copies of a file but two names for the same inode — deleting one name never deletes the data, whereas a symbolic link is just a stored path that silently breaks when its target disappears. I had assumed links behaved like Windows shortcuts, and this practical showed me the mental model was wrong.